

Frontend Theory Assignment-2

1. What is CSS Box Model?

The **CSS Box Model** describes how an HTML element is structured and how its size is calculated.

It consists of four parts:

1. **Content** – The actual text, image, or other content inside the element.
2. **Padding** – Space between the content and the border.
3. **Border** – A line surrounding the padding and content.
4. **Margin** – Space outside the border.

Example:

```
div {  
    width: 200px;  
    padding: 20px;  
    border: 5px solid black;  
    margin: 10px;  
}
```

The box model is useful for controlling the **size, spacing, and layout** of elements on a webpage.

2. What is Media Query? Explain with sample code.

A **media query** is a CSS technique used to apply different styles depending on the screen size or device characteristics.

It is mainly used to create **responsive websites** that work properly on computers, tablets, and mobile phones.

Example:

```
<style>  
  
.box {  
    width: 300px;  
    height: 100px;  
    background-color: blue;  
}  
  
@media (max-width: 600px) {
```

```
.box {  
    width: 100%;  
    background-color: green;  
}  
}  
</style>
```

```
<div class="box"></div>
```

Explanation:

When the screen width is greater than 600px, the box is blue and 300px wide.

When the screen width is **600px or less**, the media query is applied and the box becomes green and takes the available width.

3. What are the features of Bootstrap?

Bootstrap is a front-end framework used to create responsive and attractive websites quickly.

Main features of Bootstrap:

1. **Responsive design** – Websites automatically adjust to different screen sizes.
2. **Grid system** – Provides a 12-column grid system for creating layouts.
3. **Mobile-first** – Designed to work well on mobile devices.
4. **Predefined classes** – Provides ready-made CSS classes for buttons, text, colors, spacing, etc.
5. **Components** – Provides components such as navbar, cards, alerts, modals, dropdowns, and carousels.
6. **Utility classes** – Provides classes for margin, padding, display, alignment, and sizing.
7. **Cross-browser compatibility** – Helps maintain consistent appearance across modern browsers.
8. **JavaScript components** – Provides interactive components such as modals, dropdowns, and collapsible menus.

4. Write an example of basic grid structure in Bootstrap.

Bootstrap uses a **12-column grid system**.

The basic structure is:

```
<div class="container">
```

```
<div class="row">
```

```
  <div class="col-md-4">
```

```
    Column 1
```

```
  </div>
```

```
  <div class="col-md-4">
```

```
    Column 2
```

```
  </div>
```

```
  <div class="col-md-4">
```

```
    Column 3
```

```
  </div>
```

```
</div>
```

```
</div>
```

Explanation:

- container → Provides a responsive container.
- row → Creates a row.
- col-md-4 → Creates a column that occupies 4 out of 12 grid columns on medium and larger screens.
- Three col-md-4 columns make a complete row because:

4 + 4 + 4 = 12

A complete HTML example would be:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Bootstrap Grid</title>
```

```
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
    rel="stylesheet">
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
<div class="row">
```

```
<div class="col-md-4">
```

```
Column 1
```

```
</div>
```

```
<div class="col-md-4">
```

```
Column 2
```

```
</div>
```

```
<div class="col-md-4">
```

```
Column 3
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</body>
```

```
</html>
```

5. What are CSS Selectors?

CSS selectors are used to select HTML elements on which we want to apply CSS styles.

Types of CSS Selectors:

1. Element Selector

Selects all elements of a particular type.

```
p {
```

```
    color: blue;
}
```

This selects all <p> elements.

2. ID Selector

Uses # followed by the ID.

```
#heading {
    color: red;
}
```

It selects the element having id="heading".

3. Class Selector

Uses . followed by the class name.

```
.box {
    background-color: yellow;
}
```

It selects elements having class="box".

4. Universal Selector

Uses * and selects all elements.

```
* {
    margin: 0;
}
```

5. Group Selector

Used to apply the same style to multiple elements.

```
h1, p {
    color: blue;
}
```

6. Descendant Selector

Selects an element inside another element.

```
div p {
    color: green;
}
```

This selects <p> elements that are inside a <div>.

6. What are CSS Backgrounds? List the properties.

CSS background properties are used to control the **background color, image, position, size, and repetition** of an HTML element.

Main background properties:

1. **background-color** – Sets the background color.

```
body {  
    background-color: lightblue;  
}
```

2. **background-image** – Sets a background image.

```
body {  
    background-image: url("image.jpg");  
}
```

3. **background-repeat** – Controls whether the background image repeats.

```
background-repeat: no-repeat;
```

4. **background-position** – Sets the position of the background image.

```
background-position: center;
```

5. **background-size** – Controls the size of the background image.

```
background-size: cover;
```

6. **background-attachment** – Controls whether the background image scrolls with the page.

```
background-attachment: fixed;
```

7. **background** – Shorthand property used to set multiple background properties together.

7. How can we overlap elements in CSS?

Elements can be overlapped using **CSS positioning** and the **z-index property**.

Usually, we use:

- position: relative
- position: absolute
- z-index

Example:

```
<style>
```

```
.container {  
    position: relative;  
    width: 300px;  
    height: 200px;  
}
```

```
.box1 {  
    position: absolute;  
    width: 150px;  
    height: 150px;  
    background-color: red;  
    left: 20px;  
    top: 20px;  
    z-index: 1;  
}
```

```
.box2 {  
    position: absolute;  
    width: 150px;  
    height: 150px;  
    background-color: blue;  
    left: 80px;  
    top: 80px;  
    z-index: 2;  
}
```

```
</style>
```

```
<div class="container">  
    <div class="box1"></div>  
    <div class="box2"></div>  
</div>
```

Here, the two boxes overlap.

The blue box appears in front because it has a higher z-index:

z-index: 2;

while the red box has:

z-index: 1;

8. What are the different positioning properties in CSS?

The position property specifies how an HTML element is positioned.

There are **five main values**:

1. Static

This is the **default position** of an element.

```
div {  
    position: static;  
}
```

The element follows the normal flow of the webpage.

2. Relative

The element remains in the normal document flow but can be moved using top, bottom, left, or right.

```
div {  
    position: relative;  
    top: 20px;  
    left: 10px;  
}
```

3. Absolute

The element is removed from the normal document flow and positioned relative to its containing block.

```
div {  
    position: absolute;  
    top: 50px;  
    left: 100px;  
}
```

4. Fixed

The element is positioned relative to the browser viewport and generally remains in the same place when the page is scrolled.

```
div {  
    position: fixed;  
    bottom: 20px;  
    right: 20px;  
}
```

5. Sticky

The element behaves normally until a specified scrolling position is reached, after which it sticks to that position.

```
div {  
    position: sticky;  
    top: 0;  
}
```

9. What is ECMAScript?

ECMAScript is a standard or specification that defines the core rules and features of the JavaScript programming language.

JavaScript is an implementation of the ECMAScript standard.

For example, ECMAScript defines features such as:

- Variables
- Functions
- Objects
- Classes
- Loops
- Conditional statements
- Promises
- Modules

Examples of ECMAScript versions include:

- ES5
- ES6 / ECMAScript 2015
- ECMAScript 2016

- ECMAScript 2017
 - And later yearly editions
-

10. Difference between var, let, and const. Explain Temporal Dead Zone.

var, let, and const are all used to declare variables in JavaScript, but they differ in their **scope, redeclaration, and reassignment**.

var is function-scoped, which means a variable declared with var is accessible throughout the function in which it is declared. A var variable can be **redeclared and reassigned**. For example:

```
var x = 10;

var x = 20; // Allowed

x = 30;    // Allowed
```

let is block-scoped, which means it is accessible only within the block { } where it is declared. A let variable **cannot be redeclared in the same scope**, but it can be **reassigned**.

```
let x = 10;

// let x = 20; // Error

x = 30;    // Allowed
```

const is also block-scoped. A const variable **cannot be redeclared or reassigned**, and it must be initialized when it is declared.

```
const x = 10;

// x = 20;    // Error
```

All three declarations are hoisted, but their behavior differs. var is hoisted and initialized with undefined, while let and const are hoisted but remain uninitialized until their declaration is executed.

Temporal Dead Zone (TDZ)

The **Temporal Dead Zone** is the period between entering a block and the point where a let or const variable is initialized. During this period, the variable cannot be accessed.

```
console.log(x); // ReferenceError
```

```
let x = 10;
```

Here, x is in the **Temporal Dead Zone** until the statement let x = 10 is executed. Trying to access it before initialization gives a ReferenceError.